



TFAnalytics

Team Fortress 2 In-Game Item Market Analyser

Devpost-ready project story structured around the official submission prompts.

Inspiration

Team Fortress 2 has had a player-driven virtual economy for almost two decades. Cosmetics, weapons, taunts, and other items are commonly valued in refined metal and Mann Co. Supply Crate Keys. Unusual items make the market especially complex: the same hat can appear with hundreds of different particle effects, and every item-effect combination can have a different value. Non-Unusual items add thousands of variants based on quality, craftability, and other attributes.

Community resources such as backpack.tf are extremely useful for checking the price of an individual item. However, understanding how the wider market changes over time still requires players to inspect many entries, remember earlier values, and compare them manually. We saw an opportunity to complement those resources with a broader, historical view of the TF2 economy.

That inspired us to build **TFAnalytics**: a market-intelligence platform where traders, collectors, and curious players can explore current values, compare past snapshots, and understand why a market is being highlighted. Our goal is not to promise profitable trades. Instead, we want to make the available evidence easier to understand so users can make better-informed decisions.

What it does

TFAnalytics collects and analyses two complementary views of the TF2 economy:

- **Exact Unusual markets**, where one item paired with one particle effect is treated as a distinct market.
- **The wider community price guide**, covering non-Unusual item variants across different qualities and craftability states.

The website presents the latest market through headline metrics, item and effect leaderboards, largest movers, historical comparisons, and interactive Plotly charts. The Insights page adds explainable market sentiment, risk flags, opportunity screening, and confidence labels. Users can also search for a specific Unusual item-effect combination or community item variant to inspect its saved price history.

As of the 20 August 2026 snapshot, the platform tracks **41,231 priced Unusual markets across 529 effects and 771 hats**, together with **5,639 community price-guide variants representing 3,009 distinct items**.

Every result is connected to its underlying data. Visitors can browse or download the raw and processed CSV snapshots, see when each dataset was last updated, and receive a warning if an automated update becomes stale. The prices are indicative community guide values rather than guaranteed sale prices, and the risk or opportunity indicators are decision-support tools rather than financial advice.

How we built it

We built the project primarily in **Python**. Scheduled **GitHub Actions** workflows request daily price-guide data from the **backpack.tf API** using Requests. One pipeline collects exact Unusual markets, while another builds the wider community price-guide dataset without mixing in Unusual quality. A separate **Steam Web API** workflow maintains the official item-image catalogue displayed by the website.

After collection, **pandas** and **NumPy** processing code cleans the records and normalises prices into comparable refined-metal and key equivalents. Before a new snapshot is accepted, automated validators check required columns, timestamps, prices, duplicates, and unexpected row-count drops. A suspicious or malformed snapshot fails the workflow instead of silently replacing reliable data.

Accepted raw and processed snapshots are versioned as CSV files in GitHub and copied to date-stamped **Google Sheets** tabs as a secondary archive. Coordinated workflow schedules and concurrency controls prevent two automated jobs from trying to update the repository at the same time. Continuous integration compiles the Python code and runs the unit-test suite on every push to `main` and on pull requests.

The front end is a multi-page **Streamlit** application. We use **Plotly** for interactive charts, while reusable Streamlit components provide metrics, tables, story cards, confidence indicators, and consistent page headers. The application discovers the newest CSV snapshots automatically and includes the latest file signature in its cache keys so newly committed data appears without stale cached results.

We also used **Selenium** and **Beautiful Soup** for supplementary browser-assisted collection and parsing utilities. During development, we experimented with SQLite, Matplotlib, and generated PDF reports. We ultimately simplified the deployed product around versioned CSV history, Plotly visualisations, and direct data downloads because that architecture was more useful and maintainable for the finished website.

Challenges we ran into

One major challenge was turning a changing external price guide into reliable historical data. The Unusual and non-Unusual datasets have different schemas, and prices can be expressed in either keys or refined metal. We had to create separate cleaning paths while preserving enough shared structure for comparisons and trends.

Automation introduced another set of problems. External requests can fail temporarily, scheduled workflows can overlap, and a bad response should never be committed as if it were a valid market update. We added retries, strict snapshot validation, staggered schedules, shared concurrency controls, and archived diagnostic logs so failures are visible and recoverable.

We also encountered a caching bug where the deployed homepage continued showing the 7 August snapshot even after newer CSV files had been committed. The app had cached the result without a changing file-based key. We fixed this by discovering the newest snapshot first and including file signatures in the cached loaders. We then added freshness warnings and regression tests so the same problem is less likely to return.

Finally, we could not find a dependable source for Unusual particle-effect images. Rather than display inconsistent or broken imagery, we removed that feature and kept official item images from the Steam Web API. Making that scope decision improved the reliability and clarity of the final product.

Accomplishments that we're proud of

- Building a complete automated pipeline that collects, cleans, validates, archives, and publishes new market data without requiring a team member's computer to remain online.
- Combining exact Unusual markets and the wider community price guide in one interface without confusing their different meanings.
- Growing a reproducible daily history instead of showing only a single live price lookup.
- Turning more than 40,000 exact Unusual markets into interactive, searchable views that remain understandable to non-technical users.
- Making sentiment, risk, opportunity, and confidence indicators explainable rather than presenting unexplained predictions.
- Adding validation, stale-data detection, coordinated workflows, and an 18-test regression suite to protect the reliability of the application.
- Providing both an approachable website and downloadable source datasets for users who want to conduct their own analysis.

What we learned

We learned that building a trustworthy data product involves much more than creating charts. Collection, validation, freshness, caching, failure recovery, and clear definitions are just as important as the final interface. A visually convincing dashboard can still be misleading if its data is stale or if users do not understand what a metric represents.

We also learned to normalise data early. Converting refined-metal and key prices into consistent comparable values made almost every later task simpler, from snapshot comparisons to trend charts and risk indicators. At the same time, we learned not to hide uncertainty: confidence labels and data limitations are essential when markets have short histories or sparse observations.

From a product perspective, we learned that removing an unreliable feature can be better than keeping it for appearance. Dropping effect imagery, static Matplotlib graphs, and generated PDF reports allowed us to focus on the interactive experience and the automated pipeline that users actually benefit from. We also gained practical experience collaborating through Git, scheduling work with GitHub Actions, consuming external APIs, testing data workflows, and deploying a Python application through Streamlit Community Cloud.

What's next for Team Fortress 2 In-Game Item Market Analyser

The most valuable next step is to keep collecting snapshots. A longer history will make trend, volatility, and confidence indicators more meaningful and will allow us to examine seasonal changes in the TF2 economy.

As the archive grows, we would move historical data from the Git repository into durable object storage or a database and expose a small data API to the website. That would improve scalability while keeping the existing Python analytics pipeline. We would also like to add watchlists and optional alerts, improve mobile accessibility, and let users save comparisons between selected items or effects.

Longer term, we would carefully evaluate additional reliable data sources for liquidity, listing depth, and completed transactions. These would help distinguish a guide-price change from real trading activity. Any new source would need the same validation, transparency, and freshness guarantees as the current pipeline before we used it in user-facing signals.